



# OASIS Security Report

## Sensitive Data Logging Security Analysis

**Date:** 2026-06-12 00:46:10 — **Model:** llama3.2:3b

**Vulnerability:** Sensitive Data Logging

**Project:** vulnerable

**Analysis root:** `../vulnerable`

### Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

## Summary

Analyzed **8** file(s) for Sensitive Data Logging.

[Return to table of contents](#)

## Statistics

**Files analyzed:** 8    **Total findings:** 13



Medium

5

Low

1

[Return to table of contents](#)

## Files

| File                                                                                           | Similarity |
|------------------------------------------------------------------------------------------------|------------|
| /work/project/dataset/vulnerable/access-control/mycontract.sol                                 | 0.601      |
| /work/project/dataset/vulnerable/reentrancy/<br>0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol | 0.597      |
| /work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol                        | 0.597      |
| /work/project/dataset/vulnerable/reentrancy/simple_dao.sol                                     | 0.588      |
| /work/project/dataset/vulnerable/reentrancy/etherstore.sol                                     | 0.586      |
| /work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol                        | 0.578      |
| /work/project/dataset/vulnerable/ordering-attacks/<br>eth_tx_order_dependence_minimal.sol      | 0.562      |
| /work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol                           | 0.541      |

[Return to table of contents](#)

## Detailed analysis

### File 1: /work/project/dataset/vulnerable/access-control/mycontract.sol

Similarity score: 0.601

Source lines: 1-31

#### ► Finding 1.1: Sensitive Data Logging (High)

##### Vulnerable code

```
receiver.transfer(amount);
```

##### Explanation

Logs the owner's address when sending funds, potentially exposing it.

##### Impact

Disclosure of sender's address to receiver.

##### Entry point

/sendTo

##### Parameters

amount

##### Exploitation steps

- Get the owner's address from tx.origin
- Use it to phishing attack

##### Example payloads

- tx.origin = 0x12345678901234567890123456789012; receiver.transfer(10000)

##### HTTP raw requests (Burp / ZAP)

```
POST /sendTo HTTP/1.1
Host: mycontract.example.com
Content-Type: application/json
{ "amount": 10000 }
```

0

## File 2: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.597

Source lines: 1-85

### ► Finding 2.1: Sensitive Data Logging (Critical)

#### Vulnerable code

```
TransferLog = Log(_log);
```

#### Explanation

Direct logging of address and value without sanitization.

#### Impact

Disclose sensitive user data, including addresses and values.

#### Entry point

/public functions/Deposit() /

#### HTTP methods

POST

#### Parameters

\_log, \_am

#### Exploitation steps

- Attack the contract by sending a crafted Log message with a malicious address or value.

#### Example payloads

- `Log(address' 0x1234567890123456789012345678901234567890', uint256' 100')`

#### HTTP raw requests (Burp / ZAP)

```
POST /transfer HTTP/1.1
```

```
Host: example.com
```

```
Content-Type: application/json
```

```
{"log": "address\u0027 0x1234567890123456789012345678901234567890", "val": 100}
```

### ► Finding 2.2: (Medium)

**Explanation**

Deposit() logs the sender and value without proper validation.

**Impact**

Potential disclosure of user data, including sender addresses.

**Entry point**

/public functions/CashOut(uint \_am) /

**Parameters**

\_am

**Exploitation steps**

- Attack the contract by sending a large amount as cash-out value.

**Example payloads**

- 

**► Finding 2.3: (Low)****Explanation**

CashOut() logs the sender and amount without proper validation.

**Impact**

Potential disclosure of user data, including sender addresses and amounts.

**Entry point**

/public functions/CashOut(uint \_am) /

**Parameters**

\_am

**Exploitation steps**

- Attack the contract by sending a large amount as cash-out value.

**Example payloads**

- 

## File 3: /work/project/dataset/vulnerable/access-control/parity\_wallet\_bug\_1.sol

Similarity score: 0.597

Source lines: 1-225

### ► Finding 3.1: Sensitive Data Logging (Medium)

#### Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external returns  
(bytes32 o_hash); function confirm(bytes32 _h) returns (bool o_success);
```

#### Explanation

Log of execution and confirmation events contains sensitive data.

#### Impact

Sensitive user data or credentials may be logged in the event logs.

#### Entry point

/execute(address, uint, bytes)

#### Parameters

\_to, \_value, \_data

#### Exploitation steps

- Exploit by making a large number of requests to execute function with high-value parameters.

#### Example payloads

- A large amount of data

#### Secure example

```
function execute(address _to, uint _value, bytes _data) external returns  
(bytes32 o_hash) { // Log the execution event only if necessary
```

Source lines: 126-467

### ► Finding 3.1: Sensitive Data Logging: Information Disclosure (Medium)

## Vulnerable code

```
function isOwner(address _addr) constant returns (bool) { return  
m_ownerIndex[uint(_addr)] > 0; }
```

### Explanation

This function logs the owner index of an address in the contract's state variable, potentially exposing sensitive data.

### Impact

Potential disclosure of user information without consent.

### Entry point

/getOwner(uint)

### Parameters

\_addr

### Exploitation steps

- Read the owner index from the contract state.

### Remediation

Use a secure data storage mechanism, such as encryption or access control lists.

### Secure example

```
uint public m_ownerIndex[uint256] = 0; // Initialize owner index array with  
zeros.  
m_ownerIndex[uint(_addr)] = uint(_addr); // Update owner index when a new  
address is added.
```



## File 4: /work/project/dataset/vulnerable/reentrancy/simple\_dao.sol

Similarity score: 0.588

Source lines: 1-35

### ► Finding 4.1: Sensitive Data Logging (Critical)

#### Vulnerable code

```
bool res = msg.sender.call.value(amount)(  
    credit[msg.sender]-=amount;  
);
```

#### Explanation

The contract logs the value of `credit[msg.sender]` before updating it, exposing sensitive user data.

#### Impact

Disclosure of sensitive user data

#### Entry point

/withdraw(uint amount)

#### Execution path

```
Sender -> call.value(amount)() -> credit[msg.sender]-=amount  
Sender <- res
```

#### Parameters

amount

#### Exploitation steps

- Attacker calls /withdraw with a large value to overflow the `credit` storage

#### Conditions

Authenticated attacker can reach /withdraw

#### Remediation

Filter sensitive data from logs, use proper log levels, and implement secure logging practices.

## File 5: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.586

Source lines: 1-39

### ► Finding 5.1: Sensitive Data Logging (Medium)

#### Vulnerable code

```
require(msg.sender.call.value(_weiToWithdraw))
```

#### Explanation

Logs the transaction value, potentially exposing sensitive user data or credentials.

#### Impact

Disclosure of sensitive user data or credentials via log files.

#### Entry point

/withdrawFunds(uint256 \_weiToWithdraw)

#### Parameters

\_weiToWithdraw

#### Exploitation steps

- Attacker can call withdrawFunds with a high value to extract sensitive data.

#### Example payloads

- `high_value = 10000000000000 ether`

#### Conditions

Attacker can reach /withdrawFunds while authenticated.

#### Remediation

Filter transaction values from logs, use secure logging practices.

#### Secure example

```
require(balances[msg.sender] >= _weiToWithdraw)
balances[msg.sender] -= _weiToWithdraw;
```

## File 6: /work/project/dataset/vulnerable/access-control/parity\_wallet\_bug\_2.sol

Similarity score: 0.578

Source lines: 1-227

### ► Finding 6.1: Sensitive Data Logging (Critical)

#### Vulnerable code

```
function() payable { Deposit(msg.sender, msg.value); }
```

#### Explanation

The `Deposit` function logs the sender and value without proper filtering or sanitization, potentially exposing sensitive user data.

#### Impact

Disclosure of sensitive user data, such as wallet addresses and transaction amounts.

#### Entry point

/WalletEvents/Deposit

#### Execution path

```
Deposit[msg.sender, msg.value]
```

#### Parameters

msg.sender, msg.value

#### Exploitation steps

- Attackers can access sender and value data from logs.

#### Example payloads

- malicious deposit data

#### Conditions

Access to wallet logs

#### Remediation

Properly filter and sanitize sensitive data in logs.

## Secure example

```
function Deposit(address _from, uint _value) payable { emit Deposit(_from,
_value); }
```

Source lines: 1-286

### ► Finding 6.1: Unlogged Multi-Sig Owner Changes (High)

#### Vulnerable code

```
function changeOwner(address _from, address _to)
onlymanyowners(sha3(msg.data)) external {
if (isOwner(_to)) return;
uint ownerIndex = m_ownerIndex[uint(_from)];
if (ownerIndex == 0) return;
...
}
```

#### Explanation

The `changeOwner` function does not log the `\_to` address, making it vulnerable to unauthorized access if logs are enabled.

#### Impact

An attacker could exploit this vulnerability by changing an owner to gain access to funds or transactions.

#### Entry point

```
changeOwner(address _from, address _to) onlymanyowners(sha3(msg.data))
external
```

#### Execution path

```
changeOwner -> getOwner -> OwnerChanged
execute -> create -> mtxs[operation hash]
```

#### HTTP methods

POST

#### Parameters

\_from, \_to

#### Exploitation steps

- Change owner to exploit access

#### Conditions

Owner has write access to the contract.

## Remediation

Implement logging for all `onlymanyowners` function calls and ensure that logs are protected by the same authorization mechanisms as the rest of the contract.

## Secure example

```
function changeOwner(address _from, address _to)
onlymanyowners(sha3(msg.data)) public {
    if (isOwner(_to)) return;
    uint ownerIndex = m_ownerIndex[uint(_from)];
    clog(
        address(_to), \
        "Change owner to exploit access",
        "Owner has write access to the contract.");
    ...
}
```

Source lines: 96-414

### ► Finding 6.1: Sensitive Data Logging (Medium)

#### Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash) {
    // ...
}
```

#### Explanation

Sensitive data (e.g., owner addresses, operation hashes) is logged without proper filtering or masking.

#### Impact

Disclosure of sensitive user data, credentials, or security tokens via log files.

#### Entry point

/execute

#### Execution path

```
execute -> confirmAndCheck -> m_pending[_operation] -> ...
```

#### HTTP methods

POST

#### Exploitation steps

- Get the logged operation hash from the contract.
- Use the operation hash to determine which owner has confirmed it.

- Log the operation hash in a public log.

### Example payloads

- `logged_operation_hash`

### HTTP raw requests (Burp / ZAP)

```
POST /execute HTTP/1.1
Host: <contract_address>
Content-Length: 0
```

## File 7: /work/project/dataset/vulnerable/ordering-attacks/eth\_tx\_order\_dependence\_minimal.sol

Similarity score: 0.562

Source lines: 1-42

### ► Finding 7.1: Sensitive Data Logging (High)

#### Vulnerable code

```
owner.transfer(reward);
```

#### Explanation

Transfers sensitive data (reward) to owner via `transfer()` function.

#### Impact

Disclosure of reward amount and potentially owner's address.

#### Entry point

/setReward()

#### Execution path

```
owner.transfer(reward); reward = msg.value;
```

#### Parameters

reward

#### Exploitation steps

- Revoke owner's access and intercept `transfer()` call.
- Modify `reward` value before transferring it.

## File 8: /work/project/dataset/vulnerable/ordering-attacks/odds\_and\_evens.sol

Similarity score: 0.541

Source lines: 1-61

### ► Finding 8.1: Sensitive Data Logging (High)

#### Vulnerable code

```
players[tot] = Player(msg.sender, number);  
tot++;
```

#### Explanation

Log of player's address is stored without proper filtering.

#### Impact

Disclosed sensitive user data (addresses) via logs.

#### Entry point

/play(uint)

#### HTTP methods

POST

#### Parameters

number, msg.sender

#### Exploitation steps

- Obtain player's address from log
- Use obtained address to make transaction

#### Example payloads

- player's address: 0x1234567890abcdef

#### Conditions

Player sends request without authentication.

#### Remediation

Properly filter sensitive data from logs.



## Secure example

```
players[tot] = Player(msg.sender, number);  
tot++;
```

[Return to table of contents](#)

## Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

## Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)